

SMART INDIA HACKATHON 2026



TITLE PAGE

- **Problem Statement ID** – SIH26171
- **Problem Statement Title** – On-device Visual Perception for Light-weight Browser Agents
- **Theme** – Smart Automation
- **PS Category** – Software
- **Team ID** – 124701
- **Team Name (Registered on portal)** – NEXORA_

AEGISBROWSE

Privacy-Preserving Visual Browser Agent

Video walkthrough: youtu.be/heZiO6_-ZtE



Proposed solution and how it addresses the problem

A local privacy layer between the screen and the server

1 Observe privately

Raw screenshot + DOM



Transport Department eSeva

Applicant Name: Rohit Khan

Licence Number: MH29 20251234

Vehicle Number: TN20 ZS 4814

Correspondence will be sent to the address below.
233, Gandhi Marg, Kochi - 440369

Login Password: [REDACTED]

Permit Status: APPROVED

Download Permit | Print

CAPTURE AND REDACTION STAY ON THE DEVICE

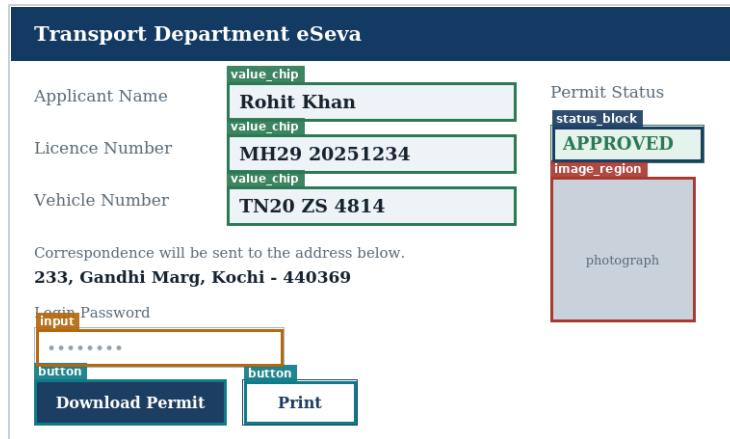
Innovation and uniqueness

Fuse privacy signals

Browser: pixels + metadata filtering

2 Locate sensitive regions

DOM signals / reference CV overlay



Transport Department eSeva

Applicant Name: Rohit Khan

Licence Number: MH29 20251234

Vehicle Number: TN20 ZS 4814

Correspondence will be sent to the address below.
233, Gandhi Marg, Kochi - 440369

Login Password: [REDACTED]

Permit Status: APPROVED

Download Permit | Print

Escalate selectively

CPU benchmark: OCR on DOM gaps

3 Prepare safe context

Redacted pixels + metadata

Illustration, not browser-model output



Transport Department eSeva

Applicant Name: [PERSON_1]

Licence Number: [LICENCE_1]

Vehicle Number: [VEHICLE_1]

Correspondence will be sent to the address below.
[ADDRESS_1]

Login Password: [REDACTED]

Permit Status: APPROVED

Download Permit | Print

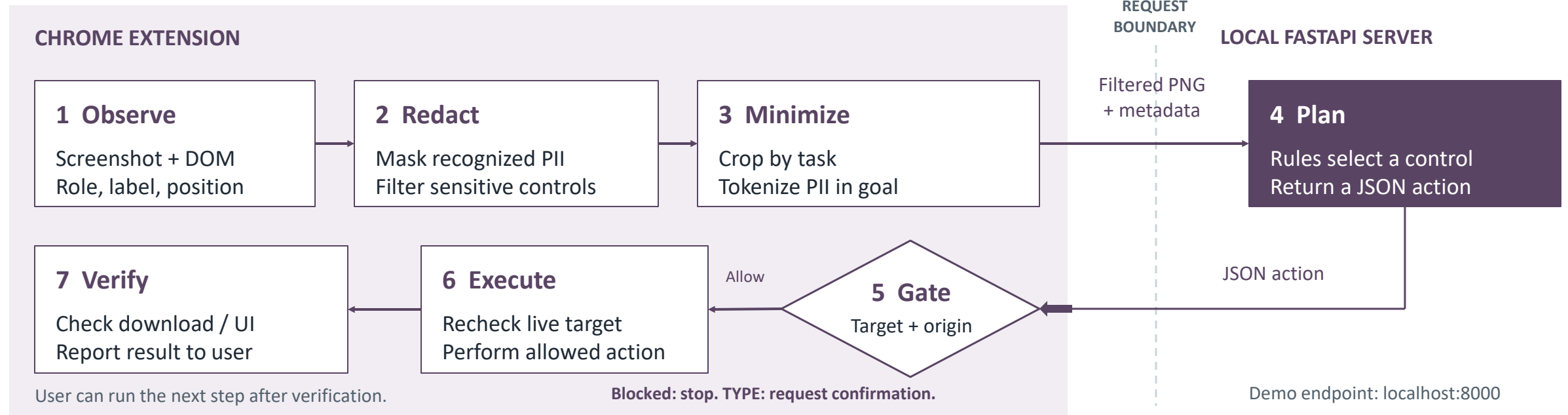
Then select only task-relevant context for the server

Disclose by task

Browser: crop + selected controls

Technologies and implementation approach

One user-triggered step, with privacy before transmission



Demo stage

DOM privacy + goal crop + guarded execution
Rules planner uses disclosed control metadata

ONNX Runtime Web: integration scaffold

Optional loader with DOM fallback is in place.
Trained detector + real runtime assets are pending.

Next: browser visual/OCR path, WebGPU / WASM timing, VLM integration tests and Firefox validation.

Analysis of feasibility

CPU reference benchmark: 48 fresh synthetic pages / 408 sensitive annotations

Criterion and measure	Always-on	Selective
Visual context (25%) UI F1 @ IoU ≥ 0.5	0.924	0.924
PII detection (20%) precision / recall	0.937 / 0.955	1.000 / 0.958
Redaction (20%) pixel precision / recall	0.643 / 0.977	1.000 / 0.973
Resources (20%) process RAM / CPU per page	57.3 MB / 52.1 ms	56.4 MB / 25.7 ms
Latency (15%) sanitize-to-plan median / p95	339 / 467 ms	47 / 48 ms

CPU reference harness, Intel Core Ultra 9 275HX: classical CV + rules planner. UI F1 is not mAP.

RAM and CPU exclude the Tesseract child process. Latency excludes browser execution and verification.

Challenges and improvement strategy

DOM-blind redaction recall: 0.878. Add label-aware OCR, then port and test selective escalation in the browser.

Still not measured

1 Trained ONNX detector

Accuracy on held-out screens

2 WebGPU timing in the browser

Cold / warm inference and memory

3 Whole-task time inside the browser

Observe through verified completion

Potential impact on the target audience

Private assistance for authenticated portal tasks

Government services, scholarships, banking KYC, health records and employee self-service.

Example: download a certificate or report

01 Protect on the device



Screen + DOM; local privacy filter

CPU harness: 48 pages, sanitized context sent and target selected

02 Plan from safe context



Filtered request; choose Download

03 Act and confirm



Guarded click; confirm download

Demo flow; completion timing pending

Benefits of the solution

PRIVACY

0

PII strings in sent metadata

SPEED

7.3×

Faster sanitize-to-plan median

COMPUTE

51%

Lower Python-process CPU / page

PLANNING

48/48

Correct Download target chosen

Details / links of reference and research work

Mind2Web	Generalist web-agent benchmark arxiv.org/abs/2306.06070
SeeAct	Grounding web actions arxiv.org/abs/2401.01614
OmniParser	Visual screen parsing arxiv.org/abs/2408.00203
PrivWeb	Local privacy filtering and controls arxiv.org/abs/2509.11939
WebPII	Visual PII benchmark and detector arxiv.org/abs/2603.17357

Prototype and demonstration

GitHub repository

github.com/mynameisizhan-coder/AegisBrowse

Extension, server, benchmark scripts and results

YouTube walkthrough

youtu.be/heZiO6_-ZtE

Prior work and our contribution

Prior work already covers local privacy filtering.
Our CPU study tests container rejection + selective OCR.
Task-aware disclosure needs a separate ablation.

Our research work: synthetic Indian-portal benchmark

12 development pages

Scholarship + HR

[Benchmark report: nexora_benchmark/SELECTIVE_RESULTS.md](#)

48 fresh test pages

4 held-out families / 408 items

48 DOM-blind variants

Reduced DOM on the same test pages

Ground truth: scorer only